#include <stdint.h>

#include <stdio.h>

#include <stdlib.h>

These give the program extra tools it needs to work with **numbers, files, and conversions**.

const int HEADER_SIZE = 44;

Absolutely. Think of the program like **copying a song while changing its volume**.

When you run:

```
./volume input.wav output.wav 2.0
```

you are telling the computer:

"Take `input.wav`, make it 2 times louder, and save the new version as `output.wav`."

Here is the code in super simple words:

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
```

These are **tools the program needs**. They help the program work with files and numbers.

```
const int HEADER_SIZE = 44;
```

A WAV audio file has **44 bytes of information at the beginning**. Think of this like the **information label** on the audio file. We don't want to change it.

```
int main(int argc, char *argv[])
```

This lets the program receive what you type in the Terminal.

For example:

```
./volume   input.wav   output.wav   2.0
    ↑          ↑           ↑         ↑
 program    old audio    new audio   volume
```

Then:

```
if (argc != 4)
```

This checks:

“Did the person give me everything I need?”

If something is missing, the program tells you how to use it correctly and stops.

Next:

```
FILE *input = fopen(argv[1], "rb");
```

This means:

Open the original audio file so I can read it.

Then:

```
FILE *output = fopen(argv[2], "wb");
```

This means:

Create a new audio file where I can save my changed audio.

Next:

```
float factor = atof(argv[3]);
```

This gets the volume number.

For example:

- `2.0` → louder
- `0.5` → quieter
- `1.0` → same volume

Then:

```
fread(header, HEADER_SIZE, 1, input);
fwrite(header, HEADER_SIZE, 1, output);
```

This means:

Copy the first 44 bytes from the old audio file to the new audio file.

We copy this part exactly. We don't change it.

Next:

```
int16_t sample;
```

A song is made up of **lots and lots of tiny pieces of sound**. Each tiny piece is called a **sample**.

Then comes the most important part:

```
while (fread(&sample, sizeof(int16_t), 1, input) == 1)
{
    sample = sample * factor;
    fwrite(&sample, sizeof(int16_t), 1, output);
}
```

In super simple words, this says:

Get one tiny piece of sound → change its volume → put it in the new file → repeat until the song is finished.

For example, if one sound sample is:

```
100
```

and your factor is:

```
2.0
```

the program does:

```
100 × 2 = 200
```

That makes that part of the sound louder.

Finally:

```
fclose(input);
fclose(output);
```

This means:

We're finished, so close both audio files.

And:

```
return 0;
```

means:

Everything worked successfully.

The easiest way to remember it

Think of the program as doing this:

1. Open the old song → 2. Create a new song → 3. Copy the audio information → 4. Read each tiny piece of sound → 5. Make it louder or quieter → 6. Save it → 7. Close everything.

So if your instructor asks, **"What does your program do?"**, you can simply say:

My program takes a WAV audio file and changes its volume. It reads the original audio, changes each sound sample using the volume factor, and saves the changed audio into a new WAV file.